ME 759 Intermediary Project Report
FlashAttention in Raw CUDA

Gaurav Batra
gbatra3@wisc.edu
Computer Science (MS)

April 22, 2026

Project Repository: https://github.com/batra98/me759-flashattention
Interactive Educational Report:
https://batra98.github.io/me759-flashattention

Abstract

Standard Transformer attention mechanisms suffer from $O(N^2)$ memory complexity. Calculating the dot product of every Query with every Key requires materializing a massive $N \times N$ matrix in High Bandwidth Memory (HBM). This intermediary report details the progress on developing a custom FlashAttention forward pass in bare-metal CUDA. By utilizing Shared Memory (SRAM) and reformulating the softmax computation, we successfully demonstrate a reduction in global memory traffic from $O(N^2)$ to $O(N)$, yielding a 1.8× speedup and a 138× reduction in memory writes at $N = 4096$ on an NVIDIA Tesla T4. Furthermore, an interactive educational website has been built to visualize the Streaming Multiprocessor (SM) memory hierarchy.

1 Introduction and Motivation

In modern Deep Learning workloads, specifically autoregressive large language models, inference is fundamentally memory-bandwidth bound rather than compute-bound [1]. The standard attention formulation materializes the full $N \times N$ attention matrix S in Global Memory (HBM). For sequence lengths like $N = 4096$, writing this intermediate matrix to global memory, only to immediately read it back to compute the Softmax denominator, completely saturates the GPU memory bus.

While libraries like cuBLAS are highly optimized, they act as black boxes. Writing a custom FlashAttention kernel from scratch provides a deep understanding of hardware utilization. The objective of this project is to implement FlashAttention [1] completely from scratch in raw CUDA C++, benchmarking its latency and using NVIDIA Nsight Compute (NCU) to analytically verify the reduction in HBM traffic on an NVIDIA GPU.

2 Methodology & Algorithm Design

2.1 The Baseline (Naive) Implementation

To form a solid experimental baseline, a standard three-kernel attention mechanism was implemented:

1. **Score Kernel:** Computes $S = QK^T/\sqrt{d}$ and saves the full $N \times N$ matrix to HBM.
2. **Softmax Kernel:** Loads rows from HBM, finds the moving maximum, exponentiates, and normalizes in a multi-pass approach over HBM lines.
3. **Output Kernel:** Loads probabilities and V from HBM, computing and writing the final output O to HBM.

This naive approach exhibits massive uncoalesced global reads and creates immense HBM thrashing.

2.2 FlashAttention (Tiled, Shared Memory) Implementation

To resolve the memory bottleneck, the algorithm was redesigned to leverage the 64 KB Shared memory (SRAM) available per SM on the Tesla T4 architecture. With chosen block sizes $B_r = 32$, $B_c = 32$, and $d = 64$, the required matrices fit comfortably into a 24 KB SRAM footprint.

2.2.1 Online Softmax and Loop Tiling

Instead of processing the entire $N \times N$ matrix at once, the Q, K, and V matrices are sliced into smaller blocks. We load a small block from global memory into SRAM and compute the attention scores.

To prevent numerical overflow without requiring the entire row, we implement the Online Softmax algorithm [3]. By maintaining a moving maximum (m) and a local normalization sum (ℓ) directly within the ultra-fast thread registers, the denominator is updated incrementally:

$$m_i^{(j)} = \max\left(m_i^{(j-1)},\ \tilde{m}_{ij}\right) \tag{1}$$

$$\ell_i^{(j)} = e^{m_i^{(j-1)} - m_i^{(j)}}\, \ell_i^{(j-1)} + e^{\tilde{m}_{ij} - m_i^{(j)}}\, \tilde{\ell}_{ij} \tag{2}$$

This allows the kernel to fuse the operations. The $O(N^2)$ logits are never written to HBM, as shown in this core CUDA block:

```
// Online rescale: completely eliminating global memory writes
float m_new = fmaxf(m_i, m_tilde);
float alpha = expf(m_i - m_new);
float beta  = expf(m_tilde - m_new);

for (int k = 0; k < d; ++k) {
    float pv_k = 0.0f;
    for (int jj = 0; jj < bc_actual; ++jj)
        pv_k += S_row[jj] * V_smem[jj][k];
    O_acc[k] = alpha * O_acc[k] + beta * pv_k;
}
m_i = m_new;
l_i = alpha * l_i + beta * l_tilde;
```

3 Experimental Setup

- **Hardware:** NVIDIA Tesla T4 (sm_75, Turing Architecture, 8.1 TFLOPs FP32).
- **Compilation:** Compiled using `nvcc` via CMake with `-O3 --extended-lambda --expt-relaxed-constexpr -lineinfo`.
- **Validation:** Root Mean Square Error (RMSE) validation ensures discrepancy $< 10^{-3}$ against the naive baseline.
- **Measurement Tooling:** `cudaEvent` for latency, and `ncu` (NVIDIA Nsight Compute) for L1TEX bytes traffic evaluation.

4 Results & Benchmarks

4.1 Execution Latency (Speedup)

FlashAttention exhibits steady and significant speedups. At $N = 4096$, latency improved from $\approx$ 27.12 ms to $\approx$ 14.96 ms, yielding a **1.8× speedup**. At $N = 8192$, latency improved from $\approx$ 112.31 ms to $\approx$ 58.70 ms. The latency is completely unblocked from the memory wall.

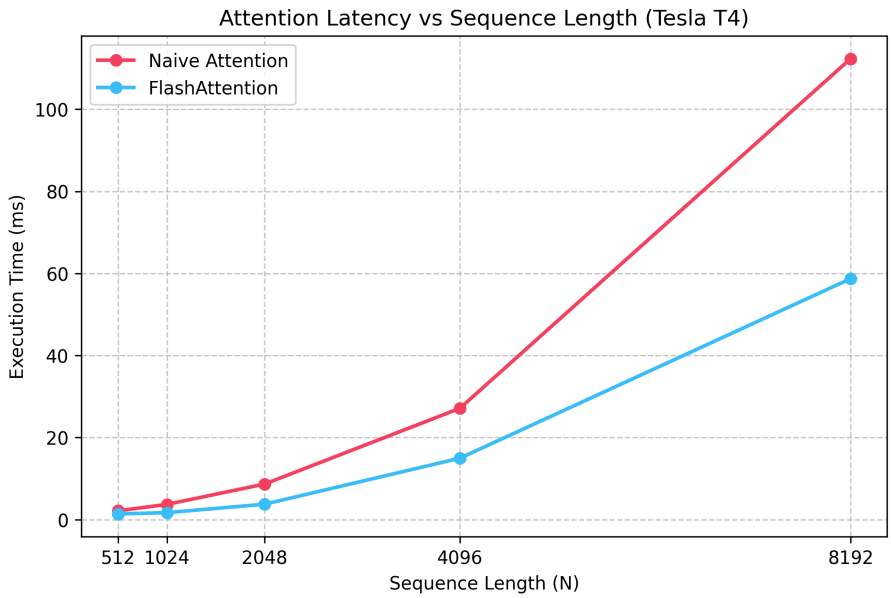

Figure 1: Execution Latency vs Sequence Length.

4.2 HBM Traffic Reduction

Nsight Compute telemetry confirms the core hypothesis. For $N = 4096$:

- **Naive Traffic:** $\approx$ 25.8 GB read, $\approx$ 1.16 GB write.
- **Flash Traffic:** $\approx$ 2.21 GB read, $\approx$ 8 MB write.

This represents an $\approx 11\times$ **reduction in reads** and an $\approx 138\times$ **reduction in writes**.

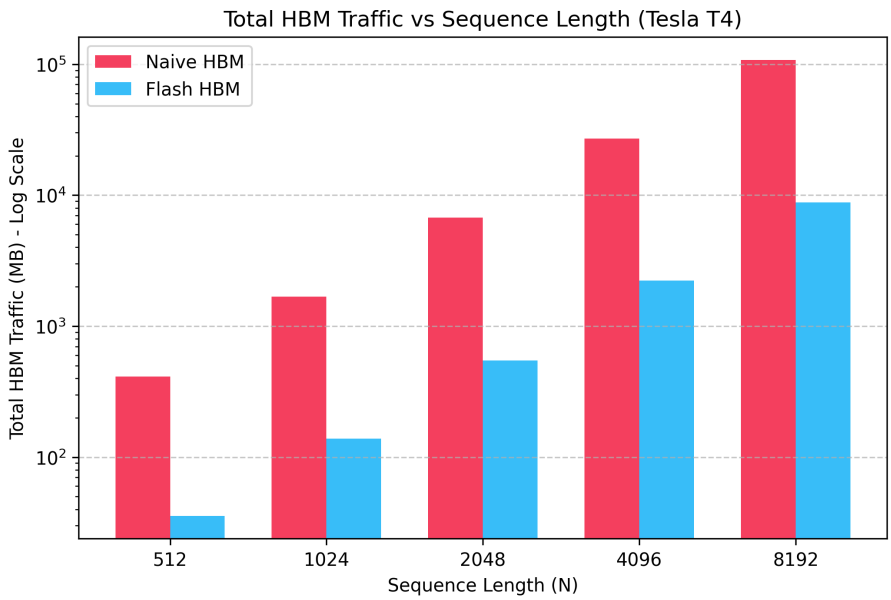

Figure 2: HBM Global Memory Traffic vs Sequence Length. Notice the quadratic scaling of the naive implementation.

5 Interactive Educational Website

Alongside the CUDA implementation, the project documentation has been transformed into a highly-readable technical blog hosted at https://batra98.github.io/me759-flashattention.

A dynamic, CSS-driven state machine visually simulates a single Streaming Multiprocessor, including a glowing Data Bus, Global Memory blocks, SRAM tiles, and a pulsing 32-thread Warp Grid. This provides users with a direct visual intuition of the $O(N^2)$ HBM bottleneck versus SRAM data locality.

6 Future Work & Final Project Trajectory

The current implementation successfully validates the FlashAttention logic in raw CUDA, achieving the theoretical $O(N)$ memory reduction. To construct the final comprehensive deliverable for ME 759, the project will implement the following advanced HPC optimizations to push the arithmetic intensity to the absolute hardware limits:

6.1 Hardware Tensor Cores (WMMA)

The current kernel utilizes standard FP32 execution units. The Tesla T4 Turing architecture features specialized Tensor Cores capable of executing mixed-precision Matrix-Multiply-Accumulate (MMA) operations incredibly fast. Future work will involve upgrading the inner dot-product computations to FP16 using the `nvcuda::wmma` API [4], mapping the block-level math natively onto Tensor Cores.

6.2 Double Buffering and `cp.async`

Currently, the SM stalls while waiting for K and V tiles to load from HBM into Shared Memory. To maximize instruction throughput, the final kernel will implement hardware

asynchronous memory copying (`cp.async`). By allocating a double-buffer in SRAM, the GPU can fetch the $(j+1)$-th tile from global memory concurrently while the compute cores process the j-th tile.

6.3 Causal Masking Integration

Autoregressive models (like GPT) only attend to past tokens, enforcing a lower-triangular causal mask on the S matrix. Engineering the thread block logic to dynamically skip fetching and computing tiles that fall outside this causal mask will effectively halve the total FLOPs required.

6.4 Warp-Level Primitives

The Online Softmax implementation currently relies on block-level shared memory reductions. To eliminate the high overhead of `__syncthreads()` barriers, the final architecture will replace these reductions with warp-synchronous `__shfl_down_sync` primitives, enabling independent 32-thread warps to resolve their maximums entirely within the register file.

6.5 Multi-GPU Distributed Attention (NCCL / MPI)

To scale beyond a single Tesla T4, the project will investigate distributing the attention mechanism across multiple GPUs. By partitioning the Query matrix Q across different GPU instances and communicating the required K and V blocks via NVIDIA Collective Communications Library (NCCL) or MPI protocols, the architecture can support massive sequence lengths that exceed the 16 GB capacity of a single GPU.

References

[1] Dao, T., Fu, D. Y., Ermon, S., Rudra, A., & Ré, C. (2022). *FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness.* NeurIPS 2022. https://arxiv.org/abs/2205.14135

[2] Vaswani, A. et al. (2017). *Attention Is All You Need.* NeurIPS 2017. https://arxiv.org/abs/1706.03762

[3] Milakov, M. & Gimelshein, N. (2018). *Online normalizer calculation for softmax.* arXiv preprint. https://arxiv.org/abs/1805.02867

[4] NVIDIA. *CUDA C++ Programming Guide: Warp Matrix Functions (WMMA).* https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html#warp-matrix-functions